MATH FOUNDATIONS

Queueing Theory Visual Guide

Little's law, the utilization-latency knee, and M/M/1 intuition for capacity planning.

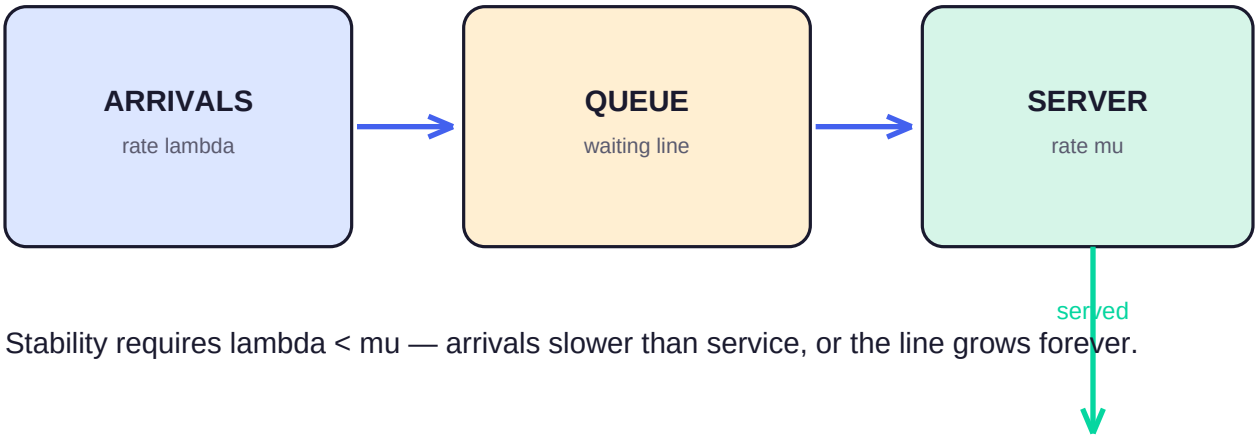
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Anatomy of a Queue

Arrivals show up, wait if busy, get served, leave.



Stability requires lambda < mu — arrivals slower than service, or the line grows forever.

Every checkout line, thread pool, and API gateway is this diagram.

STEP 2/6

Little's Law: L = lambda * W

Average number in system = arrival rate x average time in system. Always. Any system.

L = lambda * W

L: avg in system lambda: arrival rate W: avg time in system

EXAMPLE

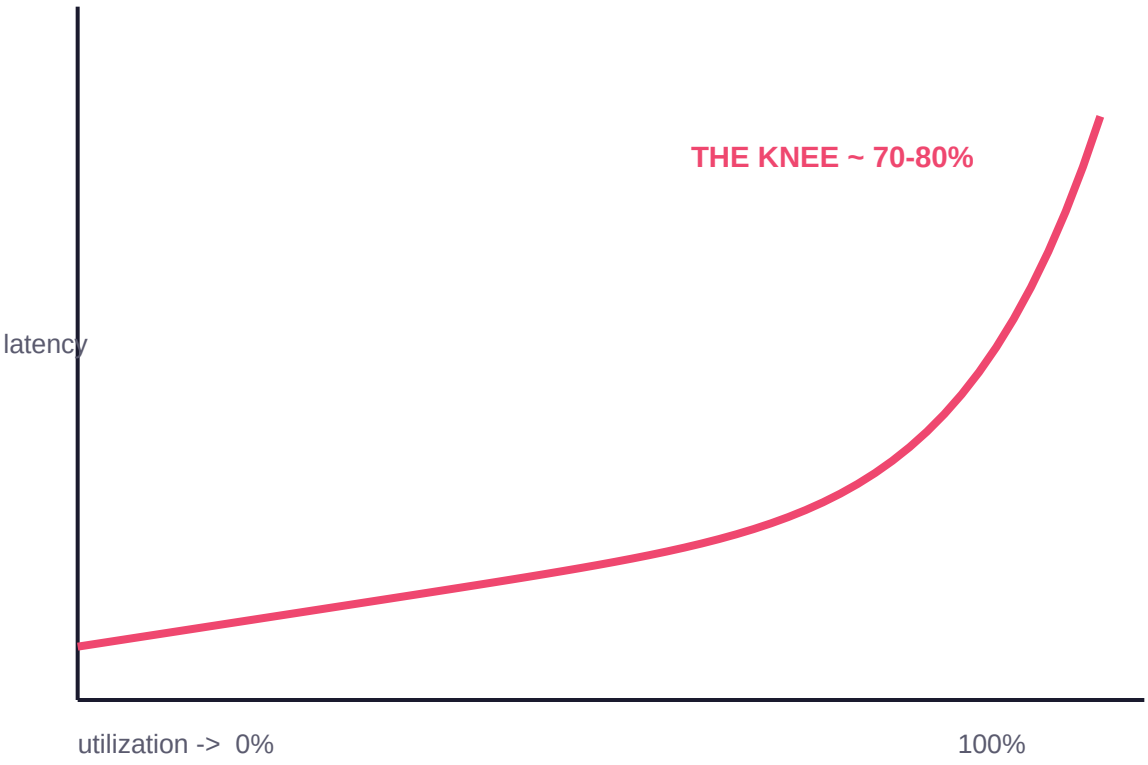
100 req/s, each spends 0.2s in system -> 20 requests in flight, always

No distributional assumptions at all — the most robust law in systems engineering. Measure two, get the third.

STEP 3/6

The Utilization vs Latency Knee

Latency explodes as utilization approaches 100%. The knee is the danger zone.



Running at 95% utilization does not give 95% efficiency — it gives unbounded queues. Plan headroom.

STEP 4/6

M/M/1 Intuition

Markov arrivals, Markov service, 1 server — the simplest non-trivial queue, with closed forms.

rho = lambda / mu

utilization: fraction of time the server is busy

L = rho / (1 - rho)

avg number in system

W = 1 / (mu - lambda)

avg time in system

At rho = 0.9: L = 9. At rho = 0.5: L = 1. Small utilization shifts, huge queue swings.

STEP 5/6

Capacity Planning Rules of Thumb

Turn the math into decisions.

Target rho <= 0.7

latency stays flat; spikes get absorbed by headroom

Size for peak + growth

measure lambda at p99 load, not the average Tuesday

Queues hide, not fix

a long queue means add capacity or shed load — not more buffering

Parallel servers help

M/M/c: c servers split load; utilization per server = lambda/(c*mu)

STEP 6/6

Queues in Real Systems

Spot the queue, apply the law.

Thread pools

fixed c servers; queue depth is your backpressure signal

Load balancers

distribute lambda across servers to keep each rho low

Backpressure

slow producers when queues fill — don't let W explode

Load shedding

when lambda > c*mu, drop or defer — protect latency for the rest

STEP ✓I✓

Key Takeaways

Queueing Theory — Remember

- A queue is arrivals (lambda), a waiting line, and service (mu); stability needs lambda < mu.
- Little's law L = lambda*W holds for any system — measure two quantities, derive the third.
- Latency explodes near 100% utilization; the knee around 70-80% is where planning lives.
- M/M/1 gives closed forms: L = rho/(1-rho), W = 1/(mu - lambda).
- Plan capacity for peak load with headroom; queues absorb bursts, they don't create capacity.
- Backpressure and load shedding protect latency when demand exceeds service.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev